MLOps Engineering Internship: Technical Assessment

Task Overview

Develop a miniature MLOps-style pipeline demonstrating the principles of reproducibility, fundamental deployment, comprehensive logging, and structured metrics output. This assessment is strictly limited to a duration of 60 minutes.

Objective

The candidate is required to create a Python application that executes deterministically, utilizing a configuration file, capable of being run as a batch job, containerized using Docker, designed to emit machine-readable metrics, and configured to log the complete execution process.

Required Command-Line Interface (CLI)

The program must adhere to the following command-line argument structure:

```
python run.py --input data.csv --config config.yaml --output metrics.json --log-file run.log
```

Configuration File Specification (`config.yaml`)

A YAML configuration file must be created containing the following precise fields:

seed: 42

window: 5

version: "v1"

Dataset Information

The candidate will be provided with a CSV file named `data.csv`, comprising 10,000 rows of cryptocurrency OHLCV (Open, High, Low, Close, Volume) data.

Column	Description	Usage
`timestamp`	Date and time of each data point	Reference
`open`	Opening price (USD)	-
`high`	Highest price in period (USD)	-
`low`	Lowest price in period (USD)	-

`close`	Closing price (USD)	**REQUIRED FOR C…**
`volume_btc`	Trading volume in BTC	-
`volume_usd`	Trading volume in USD	-

Critical Note: The `close` column must be utilized for all calculations within this assessment.

Required Logic (`run.py`)

The `run.py` script must implement the following sequential steps:

1. **Configuration Loading:**
   - Read the specified `config.yaml` file.
   - Extract the `seed`, `window`, and `version` values.
   - Set the random seed using `numpy.random.seed(seed)` or an equivalent method.
2. **Input Data Ingestion:**
   - Load the CSV file path provided by the `--input` argument.
   - Validate the file's existence and readability.
   - Ensure the presence of the required `close` column.
3. **Rolling Mean Computation:**
   - Calculate a rolling mean on the `close` column.
   - Employ the `window` size specified in the configuration (e.g., `window=5`).
   - Appropriately handle the initial rows where insufficient data exists for the rolling window calculation.
4. **Signal Generation:**
   - For each row, compare the `close` price to its corresponding rolling mean.
   - The `Signal` is assigned a value of 1 if `close > rolling_mean`.
   - The `Signal` is assigned a value of 0 if `close ≤ rolling_mean`.
5. **Metrics Calculation:**
   - Count the total number of rows successfully processed.
   - Calculate the `signal_rate`, defined as the mean of all generated signals (the proportion of 1s).
   - Measure the total program execution time in milliseconds.
6. **Robust Error Handling:**
   - The code must gracefully manage the following error scenarios:

- Missing input file.
- Invalid CSV file format.
- Empty input file.
- Missing required columns in the dataset.
- Invalid configuration file structure.

Expected Metrics Output (JSON)

A JSON file must be written to the path specified by the `--output` argument, strictly adhering to this structure:


```
{
"version": "v1",
"rows_processed": 10000,
"metric": "signal_rate",
"value": 0.4990,
"latency_ms": 127,
"seed": 42,
"status": "success"
}
```


Field	Description
`version`	Value extracted from config.yaml.
`rows_processed`	Total row count of the input dataset.
`metric`	Must always be "signal_rate".
`value`	The computed signal rate (range: 0 …
`latency_ms`	Total execution time in milliseconds.
`seed`	Value extracted from config.yaml.
`status`	"success" or "error".

Error Output Format: If an error is encountered, the output JSON must be:

```
{
  "version": "v1",
  "status": "error",
  "error_message": "Description of what went wrong"
}
```

Logging Requirements

Logs must be written to the file path specified by the `--log-file` argument and must include the following information:

1. Job Start: Timestamp and configuration loaded.
2. Configuration Verification: Confirmation of `seed`, `window`, and `version` values.
3. Data Ingestion: Number of rows loaded.
4. Processing Steps: Key operational steps (e.g., rolling mean calculation, signal generation).
5. Metrics Summary: Final `signal_rate` and `rows_processed`.
6. Job Completion: Timestamp and final status.
7. Error Reporting: Any validation errors or exceptions.

Example Log Format:

`2024-01-06` 16:00:00 - INFO - Job started

`2024-01-06` 16:00:00 - INFO - Config loaded: seed=42, window=5, version=v1

`2024-01-06` 16:00:00 - INFO - Data loaded: 10000 rows

`2024-01-06` 16:00:01 - INFO - Rolling mean calculated with window=5

`2024-01-06` 16:00:01 - INFO - Signals generated

`2024-01-06` 16:00:01 - INFO - Metrics: signal_rate=0.4990, rows_processed=10000

`2024-01-06` 16:00:01 - INFO - Job completed successfully in 127ms

Deployment Requirement: Docker (MANDATORY)

Critical: Docker containerization is a mandatory requirement. Failure to build or run the Docker container will result in the rejection of the submission.

Dockerized Batch Job Requirements:

The `Dockerfile` must:

1. Utilize a standard Python base image (e.g., `python:3.9-slim`).
2. Install necessary dependencies (`pandas`, `numpy`, `pyyaml`).
3. Copy the application code and required data files.
4. Configure the job to execute automatically upon container startup.

The Docker setup must be executable using the following exact commands:

```
docker build -t mlops-task .

docker run --rm mlops-task
```

The container environment must:

- Include `data.csv` and `config.yaml` within the image.
- Generate `metrics.json` and `run.log` during execution.
- Print the final metrics to standard output (stdout).
- Exit with a return code of 0 upon successful completion, and a non-zero code on failure.

Example Dockerfile Structure:

```
FROM python:3.9-slim

WORKDIR /app


# Install dependencies

COPY requirements.txt .

RUN pip install -r requirements.txt


# Copy application files

COPY run.py .

COPY config.yaml .

COPY data.csv .


# Run the job

CMD ["python", "run.py", "--input", "data.csv", \

    "--config", "config.yaml", "--output", "metrics.json", \

    "--log-file", "run.log"]
```

Deliverables

The submission must be a GitHub repository (or ZIP archive) containing the following mandatory files:

- ✓ `run.py`: The main Python script.
- ✓ `config.yaml`: The configuration file.
- ✓ `data.csv`: The provided dataset.
- ✓ `requirements.txt`: List of Python dependencies.
- ✓ `Dockerfile`: The container definition.
- ✓ `README.md`: Instructions for running the solution.
- ✓ `metrics.json`: An example output file (from a successful run).
- ✓ `run.log`: An example log file (from a successful run).

README Requirements

The `README.md` file must explicitly document the following sections:

1. **Setup Instructions:**
   # Install dependencies
2. pip install -r requirements.txt
3. **Local Execution Instructions:**
   # Run locally
4. python run.py --input data.csv --config config.yaml \
5. --output metrics.json --log-file run.log
6. **Docker Instructions:**
   # Build the Docker image
7. docker build -t mlops-task .
8. # Run the container
9. docker run --rm mlops-task
10. **Expected Output:** A visual representation of the `metrics.json` structure.
11. **Dependencies:** A comprehensive list of all required Python packages.

Evaluation Criteria

Criterion	Weight	Description
Correctness	40%	Rolling mean and signal generation accuracy; metrics format adherence; reproducibility.

Docker & Deployment	25%	Successful Docker build and run; correct container output; absence of hard-coded paths/parameters.
Code Quality	20%	Clean, readable structure; robust error handling; appropriate naming and commenting; adherence to Python best practices.
Logging & Observability	15%	Comprehensive and structured logging; proper error logging; clear traceability of execution flow.

Immediate Rejection Criteria

The submission will be rejected without further review if any of the following conditions are met:

- ✗ Docker build fails.
- ✗ Docker run fails or crashes.
- ✗ Non-reproducible results across multiple executions.
- ✗ Metrics are not successfully written to the JSON file.
- ✗ Hard-coded file paths or parameters are present.
- ✗ Missing run instructions in the `README.md`.
- ✗ Code fails to process the provided CSV file format.

Recommendations for Success

1. **Prioritize Functionality:** Establish the core pipeline first, then incorporate error handling and logging.
2. **Local Testing:** Validate the script using `python run.py` before initiating Dockerization.
3. **Ensure Reproducibility:** Execute the program multiple times to confirm identical outputs.
4. **Early Docker Validation:** Test the `Dockerfile` early in the development process.
5. **Edge Case Management:** Test with scenarios such as missing or empty files.

6. **Utilize Standard Logging:** The Python `logging` module is highly recommended.
7. **Maintain Clarity:** Simple, readable code is preferred over complex optimizations.

Context: MetaStackerBandit Project

This task is designed to simulate the typical work encountered on the MetaStackerBandit project, a multi-agent reinforcement learning trading system. The project leverages:

- Real-time cryptocurrency OHLCV data.
- Rolling statistics and technical indicators.
- Signal generation for trading decisions.
- Docker containers for deployment.
- Structured logging and metrics.
- Reproducible ML pipelines.

Successful completion of this assessment validates the core MLOps skills essential for the role.